Memory and processor bus, timer and counters

Yuning You

School of Science and Engineering
The Chinese University of Hong Kong, Shenzhen

Recap: Random access memory (RAM)

- RAM: The workspace
  - RAM is your computer's short-term memory.
  - It's where the computer keeps the data it is actively using so that the CPU (the brain) can access it very quickly.
  - Volatile: RAM needs power to remember. The moment you turn off your computer, everything in RAM is erased. Your desk is cleared off
  - Read and Write: The CPU can both read data from and write data to RAM very quickly
  - Fast: RAM is much faster than storage drives (like SSDs or HDDs)
  - Temporary: It holds the operating system, the applications you have open (like your web browser, Word document), and the files you are currently working on.

Recap: Read only memory (ROM)

- ROM: The permanent instructions
  - ROM is your computer's permanent, non-volatile memory
  - Non-Volatile: The data is permanently written and remains even when the computer is turned off. The instruction manual is always in the drawer
  - Read-Only (Traditionally): As the name implies, data is typically "read" from ROM and not easily written to by the user. Its contents are "hard-coded" from the factory
  - Essential for Booting: It contains the firmware, the most basic instructions for the computer

Memory Classification - RAM vs ROM Fundamentals

Volatile and non-volatile memory serve distinct roles in embedded architectures

01 **RAM (Random Access Memory)** provides fast read/write access but loses data when power is removed, making it essential for runtime variables and stack operations.

02 **ROM (Read Only Memory)** retains data permanently without power, serving as the ideal storage medium for firmware, bootloaders, and constant data tables.

03 **Selection Strategy** depends on balancing access speed requirements against data persistence needs and overall system cost constraints.

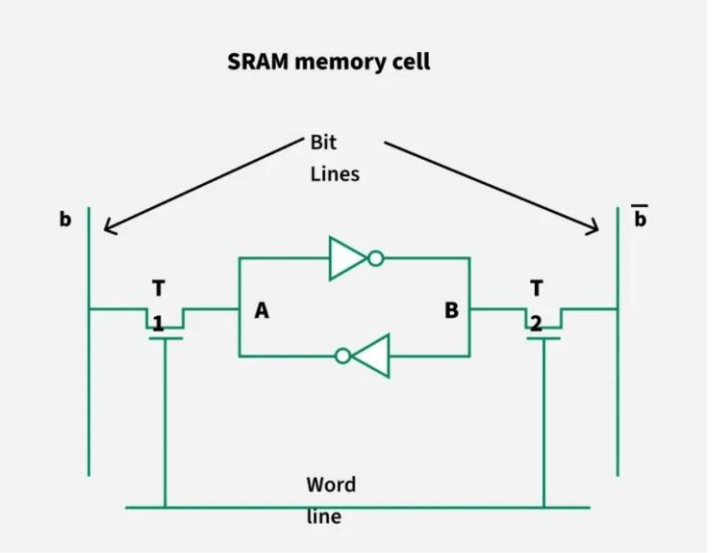
SRAM Architecture - Static Memory Cell Design

SRAM uses bistable latching circuitry for fast, low-power memory access

01 **6T Cell Design:** SRAM employs a six-transistor cell using static CMOS technology, where cross-coupled inverters form a bistable latch to hold data state.

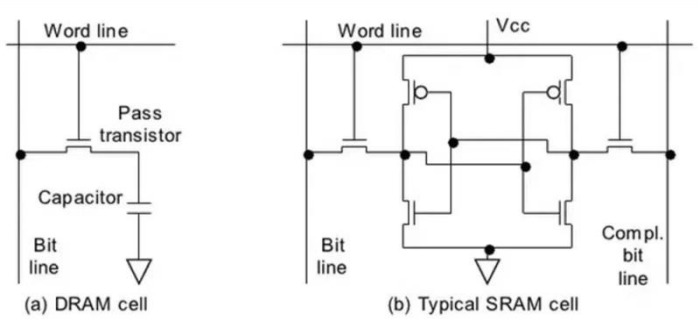
02 **Stability:** Data remains stable indefinitely as long as power is supplied, completely eliminating the need for refresh cycles and simplifying timing.

03 **Performance Tradeoff:** Offers significantly faster access times than DRAM, but at a higher cost per bit and lower density due to larger cell size.



DRAM Cell Structure - Capacitor-Based Storage

DRAM achieves high density through single-transistor capacitor cells



01 **1T1C Architecture:** Each DRAM cell consists of a single transistor and one capacitor, achieving much higher density than SRAM's six-transistor design.

02 **Charge Storage:** Data is stored as charge in the capacitor; a charged state represents logic 1, while a discharged state represents logic 0.

03 **Destructive Read:** Reading DRAM is destructive as it drains the capacitor's charge, requiring an immediate rewrite operation to restore the data.

DRAM Refresh Mechanism - Combating Charge Leakage

Periodic refresh operations prevent data loss from capacitor leakage

01

Exponential Decay

Capacitor charge exponentially decays over time due to leakage currents, typically requiring a refresh cycle every 4 milliseconds to maintain data validity.

02

Row-Level Refresh

The refresh operation reads entire rows (e.g., 256 bits) into sense amplifiers and immediately rewrites them, restoring the full charge level for all cells in that row.

03

Bandwidth Tradeoff

While essential for data integrity, the refresh overhead consumes a portion of the available memory bandwidth, as the memory cannot be accessed for read/write during refresh cycles.

Memory Array Geometry - Row and Column Organization

Two-dimensional array structure optimizes memory cell access and chip layout

01

2D Array Structure

Memory cells are arranged in large two-dimensional matrices. Row decoders select entire rows simultaneously, reducing the complexity of decoding logic compared to linear addressing.

02

Sense Amplification

Sense amplifiers are critical components that detect minute voltage differences from the memory cells and amplify them to full logic levels for reliable reading.

03

Column Selection

Once a row is active, column multiplexers select specific bytes or words from the activated row, allowing precise access to the requested data subset.

Address Multiplexing in DRAM - Reducing Pin Count

Time-multiplexed addressing halves the number of required address pins

01 **Two-Phase Addressing:** DRAM addresses are sent in two distinct phases: the row address is transmitted first, followed immediately by the column address.

02 **Control Signals:** RAS (Row Address Strobe) and CAS (Column Address Strobe) signals precisely control the timing, indicating which part of the address is currently valid on the bus.

03 **Efficiency Gain:** A 16-bit address requires only 8 physical pins instead of 16, significantly reducing package cost, size, and complexity while matching the internal array structure.

DRAM Read Cycle - Timing and Control Signals

DRAM read operations follow precise multi-phase timing protocols

01

RAS Assertion (Row Address Strobe)

The controller asserts RAS to load the row address register. This activates the entire row, latching all bits into the sense amplifiers for access.

02

CAS Assertion (Column Address Strobe)

Following the row activation, CAS is asserted to load the column address register. This enables the output buffer to drive the selected data onto the bus.

03

Critical Timing Parameters

Reliable operation depends on satisfying specific delays: RAS-to-CAS delay (tRCD) ensures row stability, and CAS access time (tCAC) defines when data becomes valid.

DRAM Write Cycle - Data Storage Protocol

Write operations merge new data into row latches before storing back to array

Phase 01

Read-Modify-Write

The target row is first read into the row latches. This step is crucial because DRAM writes often modify only a specific column within a row, requiring the rest of the row to be preserved.

Phase 02

Write Enable (WE)

The Write Enable (WE) signal is asserted along with the Column Address Strobe (CAS). This instructs the logic to overwrite the data in the selected column of the latches with the new input data.

Phase 03

Write Back

The modified contents of the row latches are then written back into the memory array, updating the charge on the capacitors and completing the storage operation.

Fast Page Mode - Optimizing Sequential Access

Techniques for rapid column access within an active row to improve memory bandwidth

01

Row Persistence

After the initial row activation, multiple column addresses can be accessed sequentially without initiating a new RAS cycle, keeping the row latches active and reducing overhead.

02

Reduced Latency

This mode significantly reduces access time for sequential or burst memory operations, as only CAS cycles are required for subsequent data retrieval within the same row.

03

Spatial Locality

Page mode effectively exploits spatial locality in memory access patterns, which is common in linear program execution and array data structure traversal.

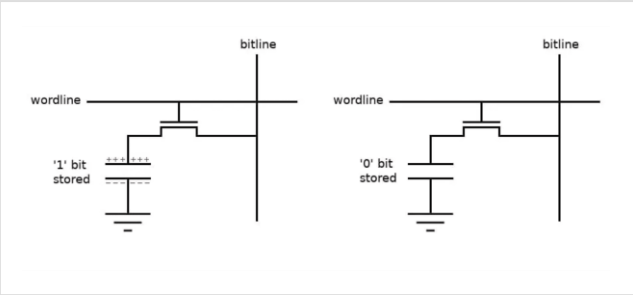
Non-Volatile Memory Technologies

Non-volatile memories retain data through power cycles for firmware and configuration

01 **Battery-Backed SRAM:** Combines the high speed of SRAM with non-volatility by integrating a battery source, maintaining data for years without external power.

02 **FRAM (Ferroelectric RAM):** Offers unlimited read/write endurance and SRAM-like performance using ferroelectric material properties, unlike Flash which wears out.

03 **Flash Memory:** Provides high-density non-volatile storage at a lower cost than alternatives, making it the standard for firmware storage despite slower write speeds.



Comparison of memory cell structures: Basis for volatile vs non-volatile technologies.

Battery-Backed SRAM - Combining Speed and Persistence

Architecture and benefits of integrating batteries with SRAM for non-volatile fast storage

01 **Integrated Power Source**

A lithium battery is molded directly into the SRAM chip package (often a "top hat" module), providing automatic backup power whenever the main system voltage fails.

02 **Zero Latency Penalty**

Unlike Flash or EEPROM, battery-backed SRAM maintains the full high-speed read/write performance of standard SRAM, with no write cycle delays or block erasure requirements.

FRAM Technology - Ferroelectric Non-Volatile RAM

Ferroelectric material properties enable fast, durable non-volatile memory

01	**Polarization State**	The ferroelectric capacitor retains its polarization state without power, with the polarization direction physically representing the stored data bit.
02	**High Endurance**	FRAM offers virtually unlimited read/write endurance compared to flash memory's limited cycles (typically 10k-100k), making it suitable for frequent data logging.
03	**SRAM Replacement**	It is intended as a non-volatile drop-in replacement for SRAM in critical applications, providing the speed of SRAM with the persistence of ROM, though currently at a higher cost.

ROM and Masked ROM - Permanent Memory Programming

Masked ROM provides immutable, high-density storage for mass-produced systems

01

Factory Programming

Data is physically encoded into the chip's photolithography masks during the manufacturing process. The bits are hardwired into the silicon structure and cannot be changed.

02

Economy of Scale

While the initial Non-Recurring Engineering (NRE) cost to create the masks is very high, the per-unit cost is extremely low, making it ideal for high-volume production.

03

Absolute Security

Since the code is physically part of the hardware, it is impossible to modify or hack via software, providing the highest level of firmware security and reliability.

PROM Types - Programmable Non-Volatile Memory

Evolution of programmable memory from OTPROM to EPROM and EEPROM

Type 01

OTPROM (One-Time Programmable)

Uses fuse or anti-fuse technology where programming physically alters the connection. Once written, the data is permanent and cannot be erased or modified.

Type 02

EPROM (Erasable PROM)

Stores charge on a floating gate. The entire chip can be erased by exposing a quartz window to strong UV light, allowing it to be reprogrammed.

Type 03

EEPROM (Electrically Erasable)

Allows individual bytes to be erased and rewritten electrically. This capability enables in-system updates and configuration changes without removing the chip.

Flash Memory Characteristics - Modern Non-Volatile Storage

High-density storage with block-based erasure architecture

01

Block Erasure

Unlike RAM or EEPROM which allow byte-level modification, Flash memory requires erasing entire blocks (typically 4KB to 64KB) before any data within that block can be rewritten.

02

High Density

Flash utilizes floating-gate transistor technology to achieve extremely high storage densities, making it the most cost-effective solution for large firmware images and data logging.

03

Endurance Limits

Flash cells degrade physically with each erase cycle, typically limiting lifespan to 10,000–100,000 cycles, which necessitates wear-leveling algorithms for data-intensive applications.

Memory Data Retention and Wearout - Reliability Considerations

Long-term stability and cycle endurance define memory lifespan

01

Data Retention

Retention refers to the ability of memory to hold data without power over time. For Flash, this is typically guaranteed for 10-20 years, but degrades significantly at higher temperatures.

02

Write Endurance

Flash memory has a finite number of program/erase cycles (e.g., 10k to 100k) before the floating gate oxide degrades, eventually leading to bit errors and write failures.

03

Wear Leveling

To mitigate endurance limits, wear leveling algorithms distribute write operations evenly across the entire memory array, preventing specific blocks from failing prematurely.

Memory Density vs Speed Tradeoffs - Design Optimization

Balancing conflicting requirements to achieve optimal system performance

01

Speed-Density Inverse

There is a fundamental inverse relationship between speed and density. High-speed SRAM requires complex 6-transistor cells, resulting in much lower density compared to the compact 1-transistor cells of slower DRAM.

02

Cost Scaling

Cost per bit increases exponentially with access speed. Designers must carefully calculate the minimum amount of expensive, high-speed memory required to meet performance targets while relying on cheaper memory for bulk storage.

03

Hierarchical Solution

To mitigate these tradeoffs, systems employ a memory hierarchy: small, ultra-fast SRAM caches feed the CPU, backed by larger, slower DRAM main memory, and finally high-density non-volatile storage.

Volatile vs Non-Volatile Comparison - Application Suitability

Guidelines for selecting appropriate memory types based on data persistence requirements

Runtime Execution

Volatile Memory (SRAM/DRAM)

Essential for variables, stack, and heap where high speed and unlimited write cycles are critical. Data loss upon power-off is acceptable as these values are re-initialized at startup.

Code Storage

Block-Based Non-Volatile (Flash)

Ideal for storing the application firmware and large static datasets. High density and low cost per bit are prioritized over write speed, as updates are infrequent.

Configuration Data

Byte-Addressable Non-Volatile (EEPROM/FRAM)

Required for calibration parameters, user settings, and error logs that must survive power cycles but change frequently enough to require byte-level modification without erasing large blocks.

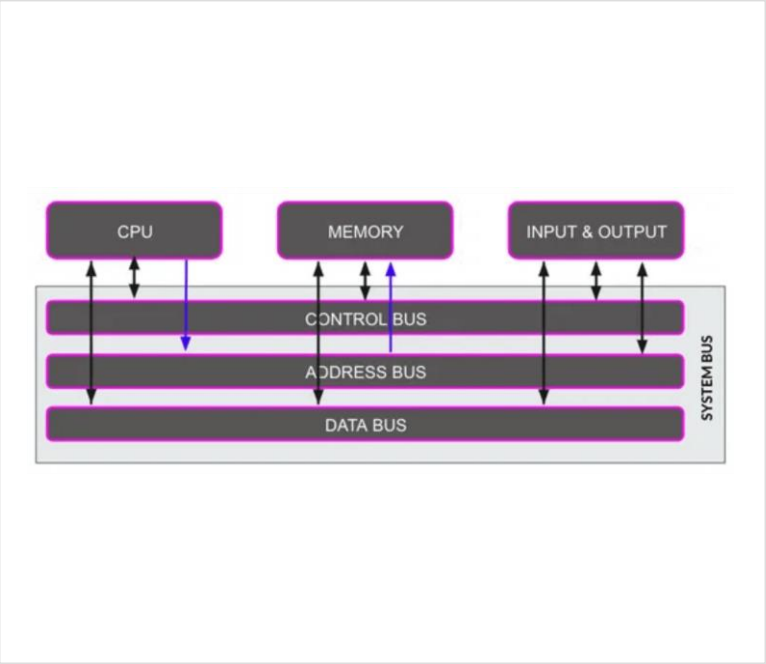
Memory Selection Criteria for Embedded Systems

Key factors driving memory choices including cost, power, speed, and environmental constraints

01	**Performance & Density**	Designers must match memory access speeds (latency/bandwidth) to CPU requirements to avoid bottlenecks, while ensuring sufficient capacity for code and data within physical size constraints.
02	**Power & Environment**	Critical for battery-operated devices; selection depends on active/sleep power consumption and the ability to operate reliably across industrial or automotive temperature ranges (-40°C to +125°C).
03	**Cost & Lifecycle**	The total Bill of Materials (BOM) cost must be minimized without compromising reliability. Long-term component availability is also crucial to support product lifecycles of 10+ years.

Memory Bus Fundamentals - Connecting CPU and Memory

The system bus serves as the critical data highway between processor and storage



01 **Shared Communication Channel:** The memory bus is a shared collection of wires that carries address, data, and control signals between the CPU, memory modules, and I/O peripherals.

02 **Master/Slave Architecture:** In a standard transaction, the CPU acts as the "Master" initiating the transfer by driving the address, while the Memory acts as the "Slave" responding with data.

03 **Von Neumann Bottleneck:** Because instructions and data share the same bus in Von Neumann architectures, the bus bandwidth often limits overall system performance, creating a processing bottleneck.

Address Bus, Data Bus, Control Bus - Three-Bus Architecture

The system bus is composed of three distinct functional groups

01

Address Bus

A unidirectional bus where the CPU asserts the physical address of the memory location or I/O device it intends to access. The bus width (e.g., 32-bit) determines the maximum addressable memory space.

02

Data Bus

A bidirectional pathway used to transfer actual data instructions and values between the CPU and memory. Its width (e.g., 64-bit) directly influences the system's data throughput and performance.

03

Control Bus

Carries command and status signals such as Read/Write, Clock, Interrupt Request, and Bus Request. These signals coordinate the timing and direction of data transfers across the other buses.

Bus Signals and Protocols - Coordinating Data Transfer

Critical timing parameters ensure valid data latching and stable communication

t_{SU}

Setup Time

The minimum amount of time that data signals must be stable and valid on the bus *before* the arrival of the clock edge or latching strobe to guarantee correct capture.

t_{H}

Hold Time

The minimum duration that data signals must remain stable *after* the clock edge or strobe transition has occurred, preventing the input from changing while being latched.

t_{ACC}

Access Time

The maximum time delay from the moment a valid address is presented on the bus to the moment valid data is available on the data lines for reading.

CPU to Memory Connections - Direct Interface Architecture

Direct parallel wiring minimizes latency but limits system expandability

Parallel Interface

The CPU connects to memory via a wide parallel bus where all address and data bits are transferred simultaneously. This requires a high pin count on the processor package (e.g., 32 address + 64 data pins).

Chip Select Logic

High-order address bits are decoded to generate Chip Select (CS) signals. These signals activate specific memory modules or banks, ensuring that only the target device drives the shared data bus.

Electrical Loading

Each memory module adds capacitance to the bus lines. In a direct connection topology, this capacitive loading limits the maximum operating frequency and the number of modules that can be connected reliably.

CPU to I/O Connections via Bus - Peripheral Communication

Integrating peripherals into the system architecture through bus interfaces

01 **Shared Bus Interface**

I/O devices physically connect to the same address, data, and control buses as the main memory. Unique address decoding logic is required to ensure that only the targeted device responds to a specific bus cycle.

02 **Memory-Mapped I/O**

Peripherals are assigned addresses within the main memory space. The CPU uses standard load/store instructions to access hardware registers, simplifying the instruction set but consuming memory address space.

03 **Port-Mapped I/O**

Uses a separate, dedicated address space for peripherals, accessed via special CPU instructions (like x86 IN/OUT). This isolates I/O from memory but requires specific processor support.

Bus Timing Diagrams Notation - Reading Waveform Specifications

Standard graphical conventions for representing digital signal states and transitions

01

Signal Transitions

Sloped lines connecting low and high levels represent rising or falling edges. These transitions are the critical trigger points for synchronous logic and latching events.

02

Data Validity

Two parallel lines indicate a valid data window where the bus is stable. Cross-hatched or crossed lines represent "don't care" or indeterminate regions where data is changing.

03

High Impedance

A single line centered between logic high and low levels indicates the High-Z (floating) state, meaning no device is currently driving the bus.

Read Cycle Timing Diagrams - Memory Read Operation

Chronological sequence of signal transitions during a memory read

t_0

Address Valid

The cycle initiates when the CPU places a stable address on the address bus. This signal must remain valid throughout the entire transaction to ensure the correct memory cell is selected.

t_1

Control Assertion

Once the address is stable, the CPU asserts the Read (RD) or Output Enable (OE) control signal active low. This triggers the memory device's internal logic to fetch data.

t_2

Data Valid

After the specified Access Time (t_{ACC}) has elapsed, the memory drives valid data onto the data bus. The CPU then latches this data on the rising edge of the control signal.

Write Cycle Timing Diagrams - Memory Write Operation

Critical timing constraints for reliable data storage

01

Address Stabilization

The address bus must stabilize *before* the Write Enable (WE) signal is asserted. This "Address Setup Time" prevents spurious writes to incorrect memory locations during signal transitions.

02

Write Pulse Width

The Write Enable signal (typically active-low) must remain asserted for a minimum duration. This pulse width ensures sufficient time for the memory internal logic to recognize the command.

03

Data Latching Edge

Data is typically latched into memory on the *rising edge* (trailing edge) of the write pulse. Data must be valid before this edge (Setup) and held stable briefly after it (Hold).

Bus Handshake Protocols - Synchronous and Asynchronous

Methods for coordinating data transfer between devices of varying speeds

01

Synchronous Protocol

All bus operations are synchronized to a common system clock. Transfers occur at fixed time intervals, offering high speed and simplicity but requiring all devices to operate at the bus frequency.

02

Asynchronous Protocol

Uses a "handshake" mechanism (Request/Acknowledge) instead of a clock. The master asserts a request, and the slave responds when ready. This is flexible but slower due to signal propagation delays.

03

Wait States

To accommodate slow peripherals on a synchronous bus, a "Ready" signal allows the slave to pause the CPU. The processor inserts "wait states" (idle clock cycles) until the device is ready to transfer data.

DMA (Direct Memory Access) Basics - CPU-Independent Transfers

Hardware mechanism allowing peripherals to access memory without CPU intervention

01

CPU Offloading

The primary goal of DMA is to free the CPU from the burden of moving large blocks of data. Instead of the CPU executing a loop of load/store instructions, the DMA controller handles the transfer autonomously.

02

Bus Mastery

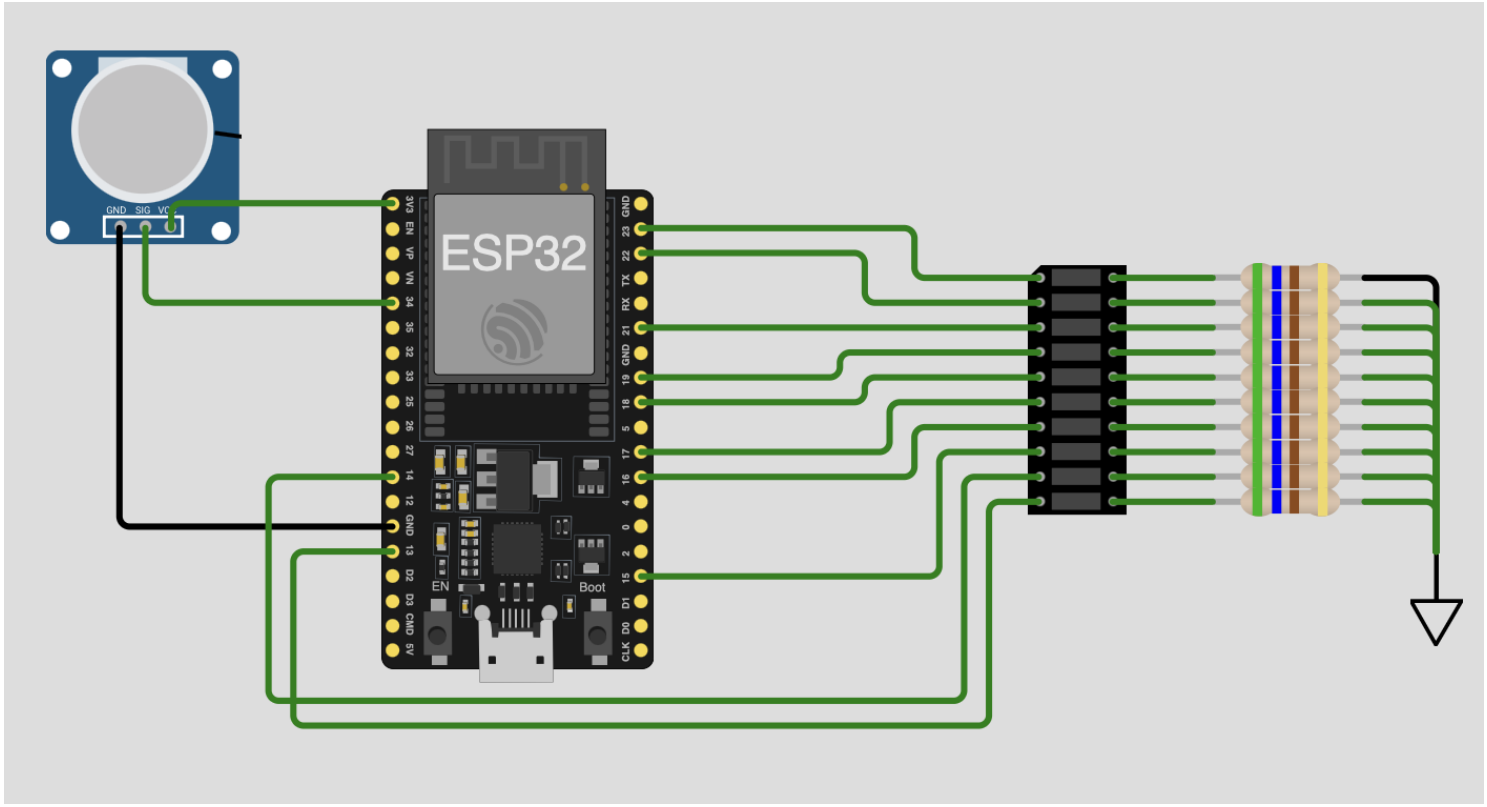
To perform transfers, the DMA controller requests control of the system bus. Once granted "Bus Master" status, it generates the necessary address and control signals to drive memory or I/O directly.

03

Interrupt Completion

The CPU configures the DMA with source, destination, and count. The DMA operates in the background and asserts an interrupt only when the entire transfer is complete, notifying the CPU to process the data.

Virtual lab: ADC-LED



Virtual lab: ADC-LED

Explain the logic of the simulation program

Virtual lab: ADC-LED

What’s the resolution of the ADC?

DMA Operation and Benefits - Efficient Bulk Data Transfer

Operational mechanics of DMA transfers and performance benefits for bulk data

01

Bus Arbitration

The DMA controller requests bus control via the HOLD signal. The CPU acknowledges with HLDA and electrically disconnects (tri-states) from the bus, handing over mastership to the DMA controller.

02

Hardware Addressing

Once in control, the DMA controller automatically drives the address bus and generates read/write control signals, incrementing the address pointer after each transfer without any software overhead.

03

Throughput Optimization

By eliminating the CPU's fetch-decode-execute cycle for every byte, DMA achieves maximum bus bandwidth, significantly speeding up large data movements like disk I/O or display updates.

Bus Arbitration Mechanisms - Sharing Bus Among Multiple Masters

Protocols for resolving contention when multiple devices request bus control

01

Mastership Contention

Since the bus is a shared resource, only one "Master" (CPU, DMA, etc.) can drive it at a time. Arbitration logic is required to decide which device gets control when simultaneous requests occur.

02

Daisy Chain Arbitration

A simple, hardware-efficient method where the "Bus Grant" signal passes serially through masters. Priority is determined by physical position, but propagation delay increases with the number of devices.

03

Centralized Parallel

A dedicated arbiter controller receives independent request lines from all masters and issues independent grant lines. This allows for flexible priority algorithms (e.g., round-robin) and faster response times.

Bus Fanout and Electrical Considerations - Signal Integrity

Electrical limitations of bus systems including capacitive loading and drive strength

01

Capacitive Loading

Every device connected to the bus adds parasitic capacitance. This increases the RC time constant, slowing down signal rise and fall times, which limits the maximum achievable bus frequency.

02

Drive Strength

The bus master (CPU or DMA) has a finite current drive capability. If the total load exceeds the fanout limit, the driver cannot source enough current to switch voltage levels quickly enough.

03

Transmission Effects

At high frequencies, the bus behaves as a transmission line. Impedance mismatches can cause signal reflections and ringing (overshoot/undershoot), potentially leading to false logic triggers.

Memory Protection Concepts - Preventing Unauthorized Access

Hardware mechanisms like MMUs for enforcing memory access permissions and isolation

Memory Management Unit (MMU)

The hardware gatekeeper that intercepts every memory access request from the CPU. It validates the virtual-to-physical address translation and checks permissions before granting access to the physical memory.

Access Permission Bits

Granular control bits assigned to memory pages (e.g., 4KB blocks) defining rights such as Read-Only (for code), Read/Write (for data), and No-Execute (NX) to prevent code injection attacks.

Process Isolation

Virtual memory addressing ensures that each process operates in its own private address space. This prevents a crashing application or malicious code from accessing or corrupting the memory of other processes or the OS kernel.

Memory-Mapped I/O vs Port-Mapped I/O - Addressing Schemes

Architectural differences in how the CPU addresses peripheral devices

Memory-Mapped I/O

Unified Address Space

I/O devices and physical memory share the same address map. A specific range of addresses is reserved for peripherals, reducing the available RAM space.

Standard Instructions

The CPU uses standard data movement instructions (e.g., LOAD, STORE, MOV) to communicate with peripherals, simplifying the instruction set architecture.

Decoding Complexity

Requires decoding the full width of the address bus (e.g., 32 bits) to distinguish between memory and I/O, potentially increasing hardware logic complexity.

Port-Mapped I/O

Isolated Address Space

I/O devices exist in a separate, dedicated address space. This preserves the full range of memory addresses for actual RAM and program code.

Special Instructions

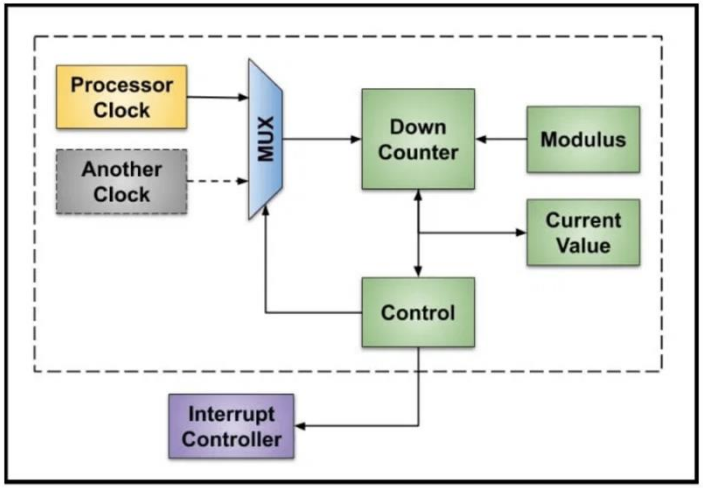
Requires dedicated CPU instructions (e.g., x86 IN, OUT) and specific control signals (IOR/IOW) to access the I/O space, distinct from memory access.

Simpler Decoding

The I/O address space is typically smaller (e.g., 16-bit), requiring fewer address lines to be decoded for peripheral selection.

Introduction to Timers and Counters - Temporal Control

Fundamentals of hardware timers and counters for time measurement and event control



Hardware Mechanism

A timer is fundamentally a digital register that automatically increments or decrements its value based on an input clock signal, operating independently of the CPU's instruction execution.

Dual Mode Operation

It functions as a "Timer" when counting fixed-frequency internal system clock pulses (measuring time) and as a "Counter" when counting asynchronous external events (measuring quantity).

Parallel Execution

By offloading timing tasks to dedicated hardware, the CPU is freed from "busy-wait" loops, allowing it to perform other processing tasks while the timer tracks time in the background.

Time as Analog vs Digital Representation - Discrete Sampling

Conceptual difference between continuous physical time and discrete digital time measurement

01

Continuous Reality

In the physical world, time is a continuous variable with infinite resolution. Events can occur at any infinitesimal moment, and duration is an analog quantity that flows without interruption.

02

Discrete Counting

Digital systems cannot represent continuity. Instead, they measure time by counting discrete events—specifically, the cycles of a fixed-frequency oscillator. Time becomes an integer count of "ticks."

03

Quantization Error

Because digital time is granular, there is always a "quantization error." An event occurring between two clock ticks is effectively rounded to the nearest tick, limiting the system's temporal resolution.

System Clock as Timing Basis - Clock Cycles and Time

Role of the system clock as the fundamental time base for all digital operations

The Digital Heartbeat

The system clock is a continuous square wave signal that oscillates between logic high (1) and logic low (0). It provides the rigid temporal structure required for the processor to execute instructions sequentially.

Edge-Triggered Synchronization

Digital circuits typically change state only on specific transitions of the clock signal, known as "edges" (Rising Edge: 0→1, Falling Edge: 1→0). This ensures that data is stable before it is latched.

Frequency vs. Period Relationship

The speed of the clock is defined by its Frequency (f), measured in Hertz (cycles per second). The duration of a single clock cycle is the Period (T), which is the inverse of the frequency.

> $T = 1 / f$ (e.g., $f = 100$ MHz → $T = 10$ ns)

Clock Cycles and Discrete Time - Quantized Temporal Resolution

Impact of clock frequency on time resolution and quantization errors

$$Res = T_{clk}$$

Resolution Limit

The system clock period defines the absolute finest granularity of time measurement. A digital timer cannot distinguish between two events that occur within the same clock cycle; they are effectively simultaneous.

$$Err \leq T_{clk}$$

Quantization Error

When an asynchronous external event occurs, it must be synchronized to the next clock edge. This introduces a variable delay (jitter) ranging from 0 to one full clock period, known as quantization error.

$$P \propto f_{clk}$$

Power vs. Precision

Increasing the clock frequency (f) improves resolution and reduces error (smaller T). However, dynamic power consumption scales linearly with frequency, creating a fundamental design trade-off.

Physical Clock Sources and Stability - Oscillator Technologies

Comparison of RC, crystal, and atomic oscillators in terms of stability and accuracy

RC Oscillator

~10,000 ppm

Uses a simple Resistor-Capacitor network, often integrated directly into the microcontroller silicon. While extremely low cost and robust against mechanical shock, it suffers from significant frequency drift due to temperature and voltage variations (1-5% error).

Crystal (XTAL)

~20-50 ppm

Relies on the piezoelectric effect of a quartz crystal vibrating at a precise mechanical resonance. This is the industry standard for system clocks, offering excellent stability and low phase noise at a moderate cost, though crystals are mechanically fragile.

Atomic / OCXO

< 0.001 ppm

Atomic clocks (Cesium/Rubidium) or Oven-Controlled Crystal Oscillators (OCXO) maintain frequency by referencing atomic transitions or keeping the crystal at a precise temperature. Used only where absolute time synchronization is critical (e.g., GPS, servers).

Crystal Oscillator Specifications - Frequency and Tolerance

Understanding crystal specs including nominal frequency, ppm tolerance, and temperature drift

01

Nominal Frequency

The specific center frequency for which the crystal is designed (e.g., 16 MHz or 32.768 kHz). This is the ideal value the crystal will oscillate at under specified load capacitance conditions at room temperature.

02

Frequency Tolerance

The maximum allowable deviation from the nominal frequency at a reference temperature (usually 25°C). It is expressed in parts per million (ppm).

Example: 16 MHz ±20 ppm
Error = ±320 Hz

03

Temperature Stability

The maximum frequency deviation over the entire operating temperature range (e.g., -40°C to +85°C). This drift is often the largest source of timing error in real-world applications.

Oscillator Drift and Accuracy - Real-World Timing Errors

Analysis of accumulated timing errors due to oscillator drift over long periods

Crystal:	100 PPM
Seconds/Day:	86,400
Error:	86,400 * 100e-6
Total Drift:	**~8.64 sec/day**

01

Parts Per Million (PPM)

Accuracy is standardly expressed in PPM. A value of ±50 PPM means that for every one million clock cycles, the oscillator may deviate by up to 50 cycles from the nominal frequency.

02

Environmental Drift

Oscillator frequency is not static. It shifts due to temperature changes (Temperature Coefficient), voltage fluctuations, and component aging over years, necessitating periodic calibration for precision applications.

03

Cumulative Impact

Small frequency errors accumulate into significant time offsets over long durations. A standard "low cost" crystal can result in noticeable clock drift within just 24 hours.

PLL (Phase Locked Loop) for Frequency Multiplication

Function of PLLs in generating high-frequency system clocks from lower-frequency crystals

Input Stage

Reference Source

The system starts with a standard, low-cost external crystal (e.g., 8 MHz). Using a lower frequency source simplifies PCB routing and significantly reduces electromagnetic interference (EMI) on the board.

Processing Stage

Feedback Loop

The PLL circuit compares the phase of the input signal with a divided version of its own output. It adjusts an internal Voltage Controlled Oscillator (VCO) until it locks onto a precise multiple (e.g., x10) of the input.

Output Stage

Core Clock Generation

The result is a stable, high-frequency system clock (e.g., 80 MHz) capable of driving the CPU core at full speed. This allows the processor to run fast while the external bus remains slow and stable.

Timer Prescaler Operation - Frequency Division

Mechanism of prescalers in dividing clock frequency to achieve longer timer intervals

The Constraint

High-Speed System Clock

Modern processors run at high frequencies (e.g., 80 MHz). At this speed, a standard 16-bit counter overflows in less than a millisecond (0.8ms), making it useless for measuring human-scale time intervals like seconds or minutes.

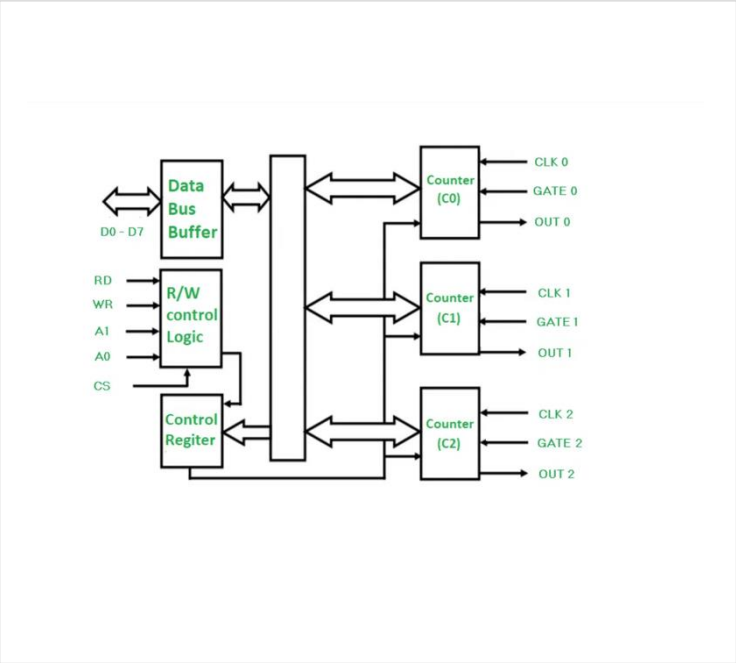
The Mechanism

Integer Frequency Division

The prescaler is a configurable hardware circuit that divides the system clock by a selectable integer factor (N) before it reaches the timer. Common ratios include 1:1, 1:8, 1:64, 1:256, and 1:1024.

Hardware Counter Architecture - Increment and Overflow

Internal structure of hardware counters, increment logic, and overflow behavior



The Counting Mechanism

At the heart of the timer is a dedicated N-bit register (e.g., 16-bit) that automatically increments its binary value by one on every rising edge of the input clock signal, independent of CPU execution.

Modulo Arithmetic

The counter operates as a modulo-2^N device. It counts linearly from 0 up to its maximum value (e.g., 65,535 for 16-bit). Upon receiving the next clock pulse, it "rolls over" back to zero.

Overflow Flag (OVF)

This rollover event is not an error but a critical timing signal. It sets a hardware Overflow Flag (OVF) and can trigger an interrupt, indicating that exactly one full timing period has elapsed.

16-bit Counter Operation and Rollover - Timing Calculations

Specifics of 16-bit counter operation, rollover periods, and range calculations

0 to 65,535

Finite Counting Range

A 16-bit counter has a fixed resolution of 2^{16} distinct states. It counts linearly from 0 (0x0000) up to its maximum value of 65,535 (0xFFFF) before running out of capacity.

0xFFFF → 0x0000

Rollover Mechanism

Upon receiving one more clock pulse after reaching the maximum, the counter wraps back to zero. This transition triggers a hardware **Timer Overflow Flag (TOV)**, which can generate an interrupt.

$T_{max} = 65536 \times T_{tick}$

Maximum Measurable Interval

The longest time duration the hardware can measure without software intervention is limited by the range. For a 1 MHz clock (1µs tick), the max period is only 65.536 ms.

Timer Frequency Calculations - Converting Counts to Time

Mathematical formulas for converting between timer counts, frequency, and real time

Timer Frequency

$$f_{timer} = f_{sys} / P$$

The effective counting frequency (f_{timer}) is derived by dividing the main system clock (f_{sys}) by the prescaler value (P). This determines how many times the timer increments per second.

Tick Period

$$T_{tick} = 1 / f_{timer}$$

The duration of a single count, or "tick" (T_{tick}), is the inverse of the timer frequency. This represents the finest temporal resolution the timer can measure.

Total Duration

$$t_{total} = N \times T_{tick}$$

The total elapsed time (t_{total}) measured or delayed is the number of counts (N) multiplied by the period of a single tick.

Converting Timer Counts to Real Time - Software Timekeeping

Software techniques for maintaining real-time clocks based on hardware timer ticks

01

Periodic Interrupt

The hardware timer is configured to overflow at a precise, fixed interval (e.g., every 1 millisecond). This generates a periodic interrupt signal that acts as the system's "heartbeat," waking the CPU to update time.

02

Tick Accumulator

An Interrupt Service Routine (ISR) increments a global counter variable. This variable must be declared as volatile to prevent compiler optimization issues.

```
volatile uint32_t sys_ticks = 0;

void Timer_ISR() {
sys_ticks++; // +1ms
}
```

03

Time Derivation

To calculate real-world time, the application software multiplies the accumulated tick count by the known interrupt period.

$$\text{Time} = \text{sys\_ticks} \times T_{\text{period}}$$

For a 1ms tick, sys_ticks = 500 simply equals 0.5 seconds.

Software Time Counters - Extending Range Beyond Hardware

Using software variables to extend the effective range of hardware timers

Hardware Constraint

A standard **16-bit hardware timer** running at 1 MHz overflows every 65.5 milliseconds. This limited range makes it insufficient for directly tracking human-scale events like seconds, minutes, or days.

+

Software Cascading

We use the hardware overflow interrupt to increment a larger variable in RAM. The hardware acts as the "millisecond hand" (fast ticks), while the software variable acts as the "hour hand" (counting overflows).

Virtual Capacity

Combining the hardware timer with a **64-bit software counter** creates a virtual timer that can run for over 500,000 years without overflowing, while maintaining the precise 1µs resolution of the hardware.

Timer Accuracy and Resolution Tradeoffs - Balancing Precision

Design tradeoffs between high resolution (fast tick) and long range (slow tick)

High Resolution

FAST CLOCK

Using a small prescaler (e.g., 1:1) results in a fast timer tick. This is ideal for measuring very short events or generating high-frequency signals.

Tick Period	1 µs
Precision	High
Max Duration (16-bit)	65.5 ms
Overflow Rate	Frequent

Long Range

SLOW CLOCK

Using a large prescaler (e.g., 1:1024) slows the tick rate. This allows the timer to run for much longer before overflowing, but fine details are lost.

Tick Period	1 ms
Precision	Low
Max Duration (16-bit)	65.5 sec
Overflow Rate	Infrequent

The Fundamental Constraint: With a fixed number of bits (N), you cannot simultaneously maximize both resolution and range. Improving one inevitably degrades the other.

$$\text{Range} = 2^N \times \text{Resolution}$$

Timer Interrupts and ISR Handling - Event-Driven Timing

Mechanism of timer interrupts and best practices for Interrupt Service Routines

01

Hardware Trigger

When the timer matches a target value or overflows, it asserts a hardware signal that immediately pauses the CPU's current execution flow. This allows the system to respond to time-critical events instantly, regardless of what the main code is doing.

02

Context Preservation

Before executing the Interrupt Service Routine (ISR), the CPU automatically pushes the Program Counter and status registers to the stack. This "context save" ensures that the main program can resume exactly where it left off once the ISR completes.

03

The "Keep It Short" Rule

ISRs must be extremely fast (microseconds). Complex processing should be deferred to the main loop by setting a flag. Long ISRs block other critical interrupts, leading to system instability and missed events.

Best Practice: Set Flag & Exit

Periodic Timer Interrupts for Scheduling - RTOS Basics

Use of periodic timer ticks for task scheduling in Real-Time Operating Systems

The System Tick

The hardware timer is configured to generate a periodic interrupt (e.g., every 1ms or 10ms). This "heartbeat" forcibly preempts the currently executing task and hands control back to the OS kernel, ensuring no single task can monopolize the CPU.

Time Slicing Logic

Upon waking, the Scheduler checks if the current task has exhausted its allocated time quantum (time slice). This mechanism, known as Round-Robin scheduling, ensures fair CPU distribution among tasks of equal priority.

Preemptive Context Switch

If a switch is required, the OS saves the full context (registers, program counter, stack pointer) of the old task and restores the context of the next task. This rapid switching creates the illusion of parallel execution on a single core.

Timer Overflow Detection - Polling vs Interrupt Approaches

Comparison of polling and interrupt-based methods for detecting timer events

Polling Method

Active Waiting

The CPU continuously reads the Timer Overflow Flag (TOV) in a software loop ("Are we there yet?"). It cannot execute other useful tasks while waiting for the flag to set.

Variable Latency

Response time depends on where the CPU is in its execution loop when the flag sets. If the loop is long, the detection may be delayed significantly.

Simplicity

Easier to implement for simple, single-task applications where power consumption and multitasking are not concerns.

Interrupt Method

Event-Driven

The CPU ignores the timer and executes other code. When the timer overflows, the hardware automatically triggers a jump to the Interrupt Service Routine (ISR).

Deterministic Response

Latency is fixed and minimal (typically a few clock cycles for context saving), ensuring precise timing regardless of the main program's state.

Efficiency

Essential for multitasking and low-power designs, as the CPU can sleep or perform other work until the exact moment the timer expires.

Multiple Timer Channels - Independent Timing Resources

Architecture of multi-channel timer peripherals for simultaneous independent timing tasks

Shared Time Base (TCNT)

A single hardware timer counter serves as the central time reference for the entire peripheral. It counts continuously based on the system clock and prescaler, providing a unified, synchronized timeline for all downstream channels.

CHANNEL A

Dedicated Compare Register

Possesses its own Output Compare Register (OCR-A). It can trigger an event at a specific count value (e.g., 1000) without affecting other channels.

CHANNEL B

Independent Interrupts

Has its own interrupt vector. The CPU can execute different code routines for Channel A and Channel B events, even if they occur within the same timer cycle.

CHANNEL C

Synchronized Parallelism

Enables the generation of phase-aligned signals. Perfect for applications like 3-phase motor control where signals must be distinct but perfectly synchronized.

Input Capture and Output Compare - Advanced Timer Modes

Advanced timer features for precise event recording and signal generation

Input Capture (IC)

A hardware mechanism that instantly "snapshots" the current timer value into a dedicated register whenever a specific edge (rising/falling) is detected on an input pin. This captures the exact time of an external event without software delay.

Primary Application

Swiss International Engineering Design System

Output Compare (OC)

A hardware mechanism that continuously compares the running timer counter against a programmable "match" value. When equality is reached, the hardware automatically toggles, sets, or clears an output pin.

Primary Application

Jitter-Free Waveform Generation (PWM)

The Hardware Advantage:

Both modes operate completely independently of the CPU. This eliminates "interrupt latency jitter," ensuring timing accuracy down to a single clock cycle even if the CPU is busy processing other tasks.

PWM Generation Using Timers - Pulse Width Modulation

Hardware generation of PWM signals for motor control and analog output

$D = t_{on} / T$

Duty Cycle Control

PWM encodes information in the time domain. The **Duty Cycle (D)** is the percentage of time the signal is HIGH within a fixed period (T). A 50% duty cycle means the signal is on for half the time and off for the other half.

$V_{avg} = V_{cc} \times D$

Analog Emulation

By switching the digital pin fast enough (typically > 1 kHz), the load (e.g., a motor or LED) integrates the pulses and responds to the **average voltage**. This allows a digital system to control analog power levels efficiently.

CPU Load ≈ 0%

Zero-Overhead Generation

The timer hardware uses "Output Compare" logic to toggle the pin automatically when the counter matches a specific value. The CPU only needs to write the duty cycle register once ("set and forget"), requiring no ongoing processing.

Watchdog Timer Introduction - Detecting Software Failures

Purpose and function of watchdog timers in system reliability and failure recovery

Definition

A Watchdog Timer (WDT) is a specialized hardware fail-safe mechanism that automatically resets the microcontroller if the software fails to periodically signal that it is operating correctly.

The Threat: System Hangs

Embedded systems are expected to run autonomously for years. However, software bugs (infinite loops, deadlocks), electrical noise (EMI), or cosmic rays can corrupt the Program Counter, causing the CPU to "freeze" or execute garbage instructions.

The Recovery: Hard Reset

The WDT acts as an independent supervisor. If the main software crashes and stops "feeding" the watchdog, the timer counts down to zero. Upon expiration, it triggers a **Hardware Reset**, forcing the system to reboot into a known good state.

Watchdog Timer Operation Principles - Timeout and Reset

Operational mechanics of watchdog timers including timeout periods and reset triggers

01

Independent Counter

The Watchdog Timer (WDT) is a hardware counter that runs continuously on a dedicated clock source (often an internal RC oscillator). This independence ensures the WDT remains functional even if the main system clock fails or the CPU halts.

02

"Kicking" the Dog

To prevent a reset, the running software must periodically write a specific sequence to the WDT control register. This action, known as "servicing," "feeding," or "kicking" the watchdog, resets the counter to its initial value.

03

Timeout & Reset

If the software hangs and fails to service the timer before it reaches zero (or overflows), the WDT hardware asserts the system RESET signal. This forces a full hardware reboot to recover from the fault.

```
if (timer == 0) {
System_Reset();
 }
```

Proper Watchdog Timer Implementation - Best Practices

Guidelines for correctly implementing watchdog servicing to ensure system safety

Rule #1

Service in Main Loop

NEVER service the WDT inside a periodic timer ISR. If the main code hangs but interrupts continue to fire (a common failure mode), the WDT will be "fooled" into thinking the system is healthy. Always kick the dog in the main background loop.

Rule #2

Windowed Operation

Use "Windowed" WDT mode if available. This enforces that the service signal must occur within a specific time window—not too late (hang), but also not too early (racing loop). This catches a wider range of software faults.

Rule #3

Task Aggregation

In multitasking systems, do not let a single task service the WDT. Implement a "safety manager" that only kicks the dog if **all** critical tasks have reported their "I'm alive" status flags within the last cycle.

Common Watchdog Timer Mistakes and Best Practices

Analysis of common watchdog implementation errors and how to avoid them

Anti-Patterns (Mistakes)

Servicing in ISRs

Placing the "kick" inside a timer interrupt is the most dangerous mistake. Interrupts often continue firing even if the main application loop has crashed, leading to a "zombie system" that never resets.

Excessive Timeouts

Setting a very long timeout (e.g., 5 seconds) "just to be safe." In control systems, leaving outputs frozen for seconds before a reset can cause physical damage or safety hazards.

Result: False sense of security with no actual protection.

Design Patterns (Best Practices)

Main Loop Servicing

Only service the WDT at the very end of the main while(1) loop. This mathematically proves that the entire application logic is cycling and completing its tasks.

Windowed Watchdog

Use a "Windowed" WDT mode if available. This enforces both a minimum and maximum time between kicks, detecting if code is running too slow (hang) OR too fast (skipping logic).

Result: Robust failure detection and rapid system recovery.